Reviewer Pack — Tropical Representation Rank of Boolean Functions Bounds AC⁰…

Entry 1792b3328f8b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 02:56:10 UTC

Tropical Representation Rank of Boolean Functions Bounds AC⁰ Parity Depth

Entry ID: `1792b3328f8b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 02:56:10 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: AC⁰ Parity Depth

Statement:

> ['For any Boolean function f with n variables, the tropical representation rank of its characteristic polynomial, τ(f), is upper-bounded by a constant c independent of n, such that for any circuit C computing PARITY on n inputs with depth d, τ(char(C)) ≤ c·log(n)·d.', 'For all instances with property P, property Q holds.']

Rationale (proposer's reasoning):

> ['Tropical geometry provides a framework to study algebraic objects over the tropical semiring. The tropical representation rank of a polynomial measures its complexity in this setting and can be computed efficiently. If such an invariant bounds AC⁰ parity depth, it could expose a new structure for proving lower bounds on circuit complexity.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `57ae43ec015a43bd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Boolean function f with n variables, if the tropical representation rank of its characteristic polynomial τ(f) is ≤ c·log(n), where c is a constant independent of n, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "AC⁰ Parity Depth" - "characteristic polynomial" tropical representation rank "Boolean function" AC⁰ Parity Depth" - "PARITY circuit depth" tropical geometry constant bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B)
    n = len(B[0])
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    Augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        pivot = Augmented[i][i]
        for j in range(i, n+1):
            Augmented[i][j] /= pivot
```

```
        for j in range(m):
            if j != i:
                factor = Augmented[j][i]
                for k in range(i, n+1):
                    Augmented[j][k] -= factor * Augmented[i][k]
    return [row[-1] for row in Augmented]

def characteristic_polynomial(f):
    n = len(f)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 1
            else:
                A[i][j] = f[j]
    return gaussian_elimination(A, [0]*n)

def tropical_representation_rank(poly):
    n = len(poly)
    max_val = -math.inf
    for term in poly:
        val = sum(abs(coeff) for coeff in term)
        if val > max_val:
            max_val = val
    return max_val

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 + (seed % 6) * 5  # Sweep n through {5, 10, 15, 20, 30, 40}
    if n < 5 or n > 40:
        return {
            "metric_name": "tropical_representation_rank",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "n_out_of_range"
        }

    f = [random.choice([0, 1]) for _ in range(n)]
    poly = characteristic_polynomial(f)
    rank = tropical_representation_rank(poly)

    c = 2  # Example constant
    d = 1  # Depth of AC⁰ circuit computing PARITY (simplified)
    upper_bound = c * math.log(n, 2) * d

    if rank <= upper_bound:
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = f"rank={rank} > {upper_bound}"

    return {
        "metric_name": "tropical_representation_rank",
```

```
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if r["instances_tested"] > 0)
    support_count = sum(1 for r in results if r["conjecture_holds"])

    mean_value = total_metric_value / len(results) if results else None
    std_deviation = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results)) / len(res
    support_fraction = support_count / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_deviation} support_fraction={support_
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_deviation} support_fraction={support_
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18f058cf.py", line 116, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18f058cf.py", line 88, in run_trial
    poly = characteristic_polynomial(f)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18f058cf.py", line 64, in characteristi
    return gaussian_elimination(A, [0]*n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18f058cf.py", line 47, in gaussian_elim
    Augmented[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed due to a division by zero error before producing data that could confirm or refute the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a valid input to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15583
2	preregistration	ollama_remote	glm4:latest	0	0	5600
3	novelty	ollama_remote	glm4:latest	0	0	4742
4	novelty	ollama_remote	glm4:latest	0	0	5680
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13393
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9435
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21630
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17432
9	judge	ollama_remote	glm4:latest	0	0	50273

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 143769 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1792b3328f8b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1792b3328f8b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1792b3328f8b.tar.gz` (if generated)